2026학년도 2학기 인문데이터 수량분석

제11강 시계열 데이터

박수민

서울대학교 인문대학 연계전공 인문데이터과학

2026년 10월 26일 월요일

오늘의 목표

1. 연·월·일 열이나 글자로 적힌 날짜를 R의 날짜·시각 자료로 바꿀 수 있다.
2. 날짜를 가로축에 놓아 그리고, 요일·시·하루 단위로 묶어 셀 수 있다.
3. 시간대와 언어 설정이 같은 자료의 결과를 어떻게 바꾸는지 설명할 수 있다.
4. 바로 앞 행과 비교하고(윈도우 함수), 기간이 겹치는 행끼리 이을 수 있다(범위 조인).

사전 준비

새로 설치할 것은 없다. `hdr/data/`에 `wiki_uk_page_views.csv`와 `wiki_uk_page_revisions.csv.bz2`가 있는지 확인한다.

```
library(tidyverse)    # 날짜를 다루는 lubridate도 함께 불러 온다
library(hms)          # 하루 안의 시각만 다룰 때 (이미 설치되어 있다)
```

1 **시간을 데이터로**
순서만 쓰기
날짜 객체

2 날짜로 묻기

3 날짜와 시각

4 언어와 시간대

5 앞뒤 행과 겹치는 기간

6 마무리

10강이 남긴 질문

한 장에 겹친 100년

10강의 교섭 네트워크는 동진 100여 년의 기록을 그림 한 장에 겹쳤다. 백제와 동진의 교섭이 372년 이후에 몰린다는 **변화**는 그 그림에서 지워진다.

시간이 붙은 인문학 자료

편지와 일기의 날짜, 실록 기사의 연월일, 신문의 발행일, 사진의 촬영 시각, 영화 장면의 시작 시각. 변화를 묻는 연구는 언제나 시간을 다룬다.

- 날짜가 적혀 있지 않아도 **순서**만 있으면 시간처럼 다룰 수 있다. 8강의 토큰 표도 문장 속 단어의 순서를 지닌다.
- 오늘은 위키백과 영국 작가 75명 페이지의 **하루 조회 수**(날짜)와 **편집 기록**(날짜와 시각)을 다룬다.

오늘의 자료 (1): 하루하루의 조회 수

```
page_views <- read_csv(
  file.path("hdr", "data", "wiki_uk_page_views.csv"))
page_views
```

```
# A tibble: 4,490 × 5
  doc_id           year month   day views
  <chr>           <dbl> <dbl> <dbl> <dbl>
1 Marie de France  2023     8     1   121
2 Marie de France  2023     8     2   138
3 Marie de France  2023     8     3   138
4 Marie de France  2023     8     4   129
5 Marie de France  2023     8     5   104
# i 4,485 more rows
```

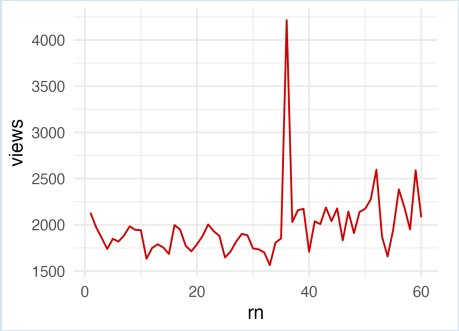
➔ 8·9강의 작가 75명 페이지마다 2023년 8월 1일부터 9월 29일까지 60일 동안의 조회 수를 모았다. 한 행이 **페이지 하나의 하루**다. 날짜는 6강의 권고대로 연·월·일을 각각의 열에 적었다.

❗ 75명 × 60일이면 4,500행이어야 한다. 10개 행은 어디로 갔는가?

행 번호를 가로축으로

```
chaucer <- page_views |>
  filter(doc_id ==
    "Geoffrey Chaucer")

chaucer |>
  mutate(rn = row_number()) |>
  ggplot() +
    geom_line(aes(x = rn,
                  y = views),
              color = "red3")
```

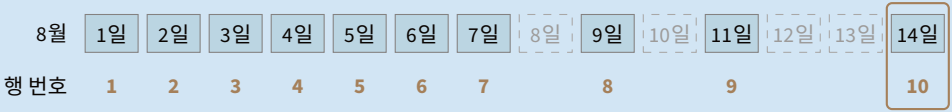


- Chaucer 페이지만 chaucer에 담아 두고 이어 쓴다. row_number()는 행마다 1, 2, 3, . . . 을 붙인다.
- 36번째 행에서 조회 수가 두 배로 뛴다. 그런데 **그날이 며칠인지** 그림만으로는 알 수 없다.

사라진 10개 행

```
page_views |>
  count(doc_id) |>     # 8강
  filter(n < 60)
```

```
# A tibble: 2 × 2
  doc_id                n
  <chr>             <int>
1 Charlotte Smith      51
2 George Bernard Shaw  59
```



⚠ Charlotte Smith 페이지는 9일이 비어 있다. 그래서 **10번째 행이 8월 14일**이다. Shaw는 마지막 날이 없다. $9 + 1 = 10$행이다.

❗ 행 번호가 날짜 노릇을 하려면 하루에 한 행씩, **빠진 날 없이** 적혀 있어야 한다.

소수 연도: 날짜를 수 하나로

교재의 어림셈

$$\text{year_frac} = \text{year} + \frac{\text{month} - 1}{12} + \frac{\text{day} - 1}{12 \cdot 31}$$

2020년 7월 1일은 $2020 + 6/12 = 2020.5$. 한 달을 31일로 치는 어림셈이다.

좋은 점

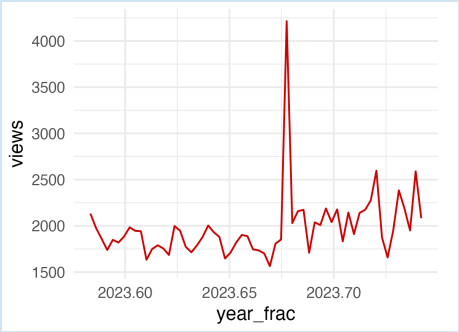
- 빠진 날이 있어도 제자리에 놓인다.
- 그냥 수라서 어떤 계산에도 넣는다.

아쉬운 점

- 2023.65가 며칠인지 읽히지 않는다.
- 요일 같은 달력 정보가 없다.

소수 연도로 그리기

```
chaucer |>
  mutate(year_frac = year +
    (month - 1) / 12 +
    (day - 1) / (12 * 31)) |>
  ggplot() +
    geom_line(aes(x = year_frac,
                  y = views),
              color = "red3")
```



- 이제 가로축이 시간이다. 조회 수가 뛴 날은 2023.68 무렵이다.
- 그래서 그날은 며칠인가? 식을 거꾸로 풀어야 9월 5일이라는 답이 나온다.
  컴퓨터에게는 편하지만 **사람이 읽기에는** 여전히 불편하다.

날짜 자료형 <date>

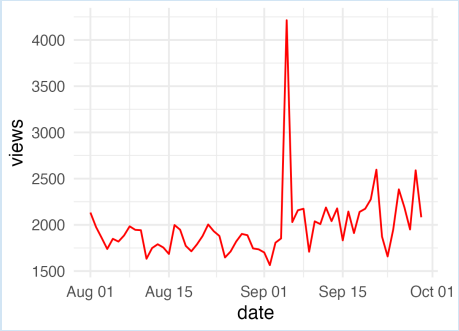
```
chaucer <- chaucer |>
  mutate(date = make_date(year, month, day))
chaucer
```

```
# A tibble: 60 × 6
  doc_id            year month   day views date
  <chr>            <dbl> <dbl> <dbl> <dbl> <date>
1 Geoffrey Chaucer  2023     8     1  2133 2023-08-01
2 Geoffrey Chaucer  2023     8     2  1977 2023-08-02
3 Geoffrey Chaucer  2023     8     3  1860 2023-08-03
4 Geoffrey Chaucer  2023     8     4  1739 2023-08-04
5 Geoffrey Chaucer  2023     8     5  1849 2023-08-05
# i 55 more rows
```

- ➔ `make_date()`는 `lubridate`의 함수다. 연·월·일 세 열을 받아 <date> 열 하나를 만든다.
- ➔ 날짜는 수와 닮았다. 순서가 있고, 간격을 잴 수 있고, 연속된 축에 놓인다. 그러면서도 요일처럼 **달력에서만 나오는 정보**를 품고 있다.
- 💡 2강에서 "뒤에서 다룬다"고 미뤄 둔 자료형이 이것이다.

날짜를 가로축에 놓으면

```
chaucer |>
  ggplot() +
    geom_line(aes(date, views),
              color = "red")
```



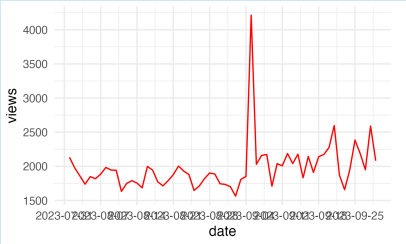
바뀐 것은 열 하나

가로축에 date를 주었을 뿐이다. ggplot이 <date>를 알아보고 Aug 01, Aug 15처럼 이름표를 단다.

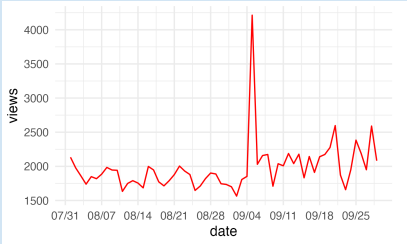
- ➔ 조회 수가 뛴 날이 9월 5일(화요일)임이 바로 읽힌다. 4,214회로 평소의 두 배다.
- ⚠ 그날 무슨 일이 있었는지는 조회 수 자료만으로는 알 수 없다. 위키백과 바깥의 기록을 찾아봐야 한다.

눈금 간격과 이름표

```
scale_x_date(
  date_breaks = "1 week")
```




```
scale_x_date(
  date_breaks = "1 week",
  date_labels = "%m/%d")
```



- 앞 장 코드 끝에 +로 붙인다. `date_breaks`는 눈금 간격, `date_labels`는 이름표의 형식이다.
- 교재 코드(왼쪽)는 이름표가 겹친다. `%m`은 월, `%d`는 일이다(3절에서 모아 본다).

날짜끼리 빼고 더하기

```r
make_date(2026, 11, 16) -
  make_date(2026, 10, 26)
make_date(2026, 10, 26) + 7
as.numeric(
  make_date(2026, 10, 26))
make_date(2023, 2, 30)
```

```
Time difference of 21 days

[1] "2026-11-02"

[1] 20752

[1] NA
```

날짜의 속

R은 날짜를 1970년 1월 1일로부터 **며칠째**인지로 적어 둔다. 오늘은 20,752일째다. 그래서 빼고 더할 수 있다.

달력에 없는 날

2월 30일은 NA가 된다. **경고도 없이.**

➔ 오늘부터 기말고사(11월 16일)까지 21일, 일주일 뒤는 13강이 있는 11월 2일이다. 6강에서 미뤄 둔 간격 계산이다.

1 시간을 데이터로

2 **날짜로 묻기**
요일과 기간
글자를 날짜로

3 날짜와 시각

4 언어와 시간대

5 앞뒤 행과 겹치는 기간

6 마무리

요일별 평균

```
chaucer |>
  mutate(weekday = wday(
    date, label = TRUE)) |>
  group_by(weekday) |>
  summarize(views_mean =
    mean(views))
```

```
# A tibble: 7 × 2
  weekday views_mean
  <ord>        <dbl>
1 Sun          1866.
2 Mon          1976.
3 Tue          2273.
4 Wed          1988.
5 Thu          2099.
6 Fri          1828.
7 Sat          1762.
```

wday()

날짜에서 요일을 꺼낸다. `label = TRUE`가 없으면 1-7의 수다.

➔ 화요일이 가장 많고 토요일이 가장 적다(교재의 결론). `<ord>`는 순서 있는 요인(4강)이라 요일이 달력 순으로 놓인다.

⚠ 교재 출력은 월요일부터지만 실제로는 **일요일부터** 나온다. 월요일부터 놓으려면 `week_start = 1`.

정말 화요일인가

```
# A tibble: 7 × 4
  weekday     n  mean median
  <ord>   <int> <dbl>  <dbl>
1 Sun         8 1866.  1820.
2 Mon         8 1976.  1894.
3 Tue         9 2273.  2133
4 Wed         9 1988.  1977
5 Thu         9 2099.  1947
6 Fri         9 1828.  1773
7 Sat         8 1762.  1713
```

하루가 끌어올린 평균

9월 5일(화) 하루가 4,214회다. 그날을 빼면 화요일 평균은 2,030으로 **목요일(2,099)보다 낮다**.

중앙값으로 보면

화요일이 여전히 가장 높지만(2,133) 차이는 줄어든다.

➔ 앞 장의 `summarize()`에 `n()`과 `median()`(중앙값)을 더했다.

❗ 요일마다 8–9일뿐이다. 하루가 순위를 바꾸는 자료로 “화요일에 많이 본다”고 단정하기는 어렵다.

날짜로 거르기

```
chaucer |> filter(date > make_date(2020, 1, 15)) |> nrow()
chaucer |> filter(date >= make_date(2023, 9)) |> nrow()
```

```
[1] 60
[1] 29
```

교재의 예

2020년 1월 15일 **뒤**를 골랐는데 60행이 그대로다. 교재를 쓰는 사이 자료가 2023년 것으로 바뀌었기 때문이다.

일부만 적어도 된다

`make_date(2023, 9)`는 9월 1일이다. 일을 빼면 그달 1일, 월까지 빼면 그해 1월 1일이 된다.

⚠ 아무것도 걸러지지 않아도 R은 말이 없다. 걸러진 결과가 **몇 행인지** 늘 확인한다.

💡 기간은 4강의 `between()`으로도 고른다. `between(date, make_date(2023, 9), make_date(2023, 9, 7))`은 9월 첫 주다.

글자를 날짜로: ymd()

```
1 ymd("2026-10-26")
2 ymd("20261026")
3 ymd("2026년 10월 26일")
4 mdy("October 26, 2026")
5 dmy("26/10/2026")
```

```
[1] "2026-10-26"
[1] "2026-10-26"
[1] "2026-10-26"
[1] "2026-10-26"
[1] "2026-10-26"
```

함수 이름이 곧 적힌 순서

y는 연, m은 월, d는 일. 사이에 낀 기호(-, /, 공백, 년·월·일)는 알아서 건너뛴다.
시각까지 있으면 ymd_hms("2026-10-26 10:30:00") 처럼 뒤에 붙인다.

- 6강의 03/04/2026: dmy()는 4월 3일, mdy()는 3월 4일로 읽는다. 어느 함수를 쓸지는 **자료를 만든 사람의 관습**을 알아야 정할 수 있다.

읽지 못한 날짜

```
dates <- c("2026-10-26", "2026.10.26", "26-10-2026")
ymd(dates)
```

```
[1] "2026-10-26" "2026-10-26" NA
Warning message:
 1 failed to parse.
```

오류가 아니라 경고

순서가 다른 셋째 것은 NA가 되고 경고가 뜬다. 오류가 아니므로 코드는 멈추지 않는다.

⚠ 경고를 지나치면 그 행은 이후의 집계에서 소리 없이 빠진다(6강의 결측). `sum(is.na(ymd(dates)))`로 몇 개인지 세어 둔다.

➔ 한 열에 형식이 섞여 있으면 후보 순서를 여럿 준다. `parse_date_time(dates, orders = c("ymd", "dmy"))`는 셋을 모두 읽는다.

함수가 모르는 달력

연호

"메이지 9년"은 1876년이다. 메이지 원년이 1868년이기 때문이다. `ymd()`는 연호를 모른다.

음력

조일수호조규(강화도조약)를 맺은 1876년 2월 27일은 조선의 날짜로 고종 13년 **음력** 2월 3일이다.

양력으로 바꾼 날

조선은 음력 1895년 11월 17일을 1896년 1월 1일로 삼으며 양력으로 바꾸었다. 그 전의 국내 기록은 대개 음력이다.

- R의 <date>는 **양력**(그레고리력) 하나만 안다. 음력이나 연호로 적힌 기록은 먼저 양력으로 옮겨야 다른 자료와 나란히 놓을 수 있다.
- 음력 날짜는 한국천문연구원의 음양력 변환표처럼 이미 정리된 대조표를 써서 옮긴다.

연호는 표를 만들어 푼다

```
eras <- tribble(
  ~era,      ~first_year,
  "메이지", 1868,
  "다이쇼", 1912)

tibble(era = "메이지",
       era_year = 9) |>
  left_join(eras,
            by = "era") |>
  mutate(year = first_year +
           era_year - 1)
```

```
# A tibble: 1 × 4
  era    era_year first_year  year
  <chr>     <dbl>      <dbl> <dbl>
1 메이지        9       1868  1876
```

5강의 조인

연호마다 원년을 적은 표를 만들고, 연호 이름을 키로 붙인 뒤 셈을 한다.

➔ 원년이 1년이므로 1을 뺀다. 메이지 9년 $= 1868 + 9 - 1 = 1876$.

1 시간을 데이터로

2 날짜로 묻기

3 **날짜와 시각**
시각까지 적힌 자료
단위 바꾸기

4 언어와 시간대

5 앞뒤 행과 겹치는 기간

6 마무리

오늘의 자료 (2): 편집 기록

```
page_revisions <- read_csv(file.path(
  "hdr", "data", "wiki_uk_page_revisions.csv.bz2"))
page_revisions
```

```
# A tibble: 35,470 × 5
  doc_id          user              datetime             size comment
  <chr>           <chr>             <dttm>              <dbl> <chr>
1 Marie de France YurikBot          2006-07-06 23:24:54  3061 robot  Adding: [[…
2 Marie de France YurikBot          2006-07-24 18:39:08  3085 robot  Adding: [[…
3 Marie de France Ccarroll          2006-09-10 13:36:44  3085 <NA>
4 Marie de France 63.231.20.88      2006-09-18 04:59:53  3105 <NA>
# i 35,466 more rows
```

- ➔ 페이지마다 **가장 최근의 편집 500번**이다. 누가, 언제 고쳤고, 고친 뒤 몇 바이트였으며(`size`), 무엇을 했다고 적었는지(`comment`).
- ➔ `<dttm>`은 날짜와 시각을 함께 담는 자료형이다. ISO 8601 형식이라 `read_csv()`가 알아본다.
- 💡 `.bz2`는 압축 파일이라는 뜻이다. 교재 본문의 파일 이름에는 빠져 있다.

500번이 커버하는 기간

```
page_revisions |>
  group_by(doc_id) |>
  summarize(n = n(), first = min(datetime)) |>
  arrange(first) |>
  slice(c(1:3, 73:75))   # 가장 이른 셋, 가장 늦은 셋
```

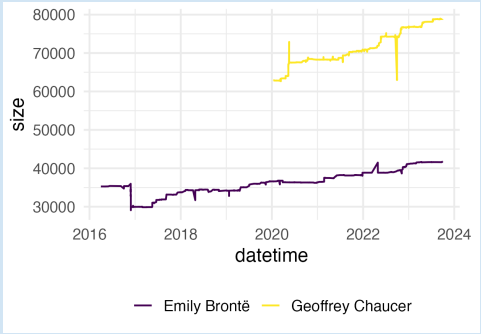
```
# A tibble: 6 × 3
  doc_id              n first
  <chr>           <int> <dttm>
1 Henry Vaughan     389 2002-10-26 10:14:44
2 Felicia Hemans    347 2003-01-04 22:59:12
3 John Gower        305 2003-01-29 04:12:18
4 Oscar Wilde       500 2021-02-17 21:02:52
5 James Joyce       500 2021-11-04 07:39:06
6 George Orwell     500 2021-11-09 03:52:19
```

➔ 자주 고쳐지는 페이지일수록 500번이 **빨리** 찬다. Orwell은 2년이 채 안 된다.

⚠ 그 전에 아무도 안 고친 것이 아니라 **자료를 모은 방식**의 차이다.

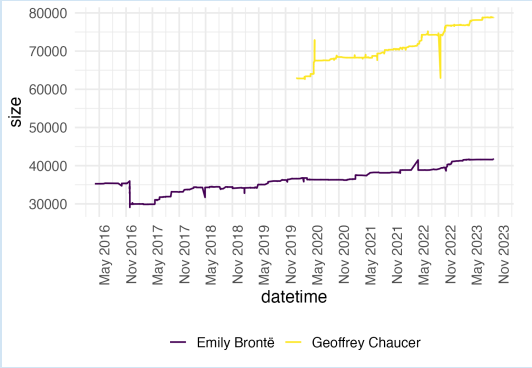
편집 뒤 페이지 크기

```
page_revisions |>
  filter(doc_id %in% c(
    "Geoffrey Chaucer",
    "Emily Brontë")) |>
  ggplot(aes(datetime,
             size)) +
  geom_line(
    aes(color = doc_id)) +
  scale_color_viridis_d()
```



- <dttm> 열도 가로축에 그대로 놓인다. Brontë는 2016년, Chaucer는 2020년부터다.
- 2022년 가을, Chaucer의 선이 뚝 떨어졌다 곧바로 돌아온다. 무슨 일인지는 5절에서 밝힌다.

이름표 형식 바꾸기




```
scale_x_datetime(
  date_breaks = "6 months",
  date_labels = "%b %Y") +
theme(axis.text.x =
  element_text(angle = 90,
               hjust = 1))
```

코드	뜻	예
%Y-%m-%d	연-월-일	2023-09-05
%b	달 이름 약자	Sep
%a	요일 약자	Tue
%H:%M	시:분	15:14

➔ `theme()` 줄은 이름표를 90도 돌린다. 형식 코드 전체는 `?strptime`에 있다.

시각으로 거르기

```
page_revisions |>
  filter(doc_id %in% c("Geoffrey Chaucer", "Emily Brontë")) |>
  filter(datetime > make_datetime(2021))
```

```
# A tibble: 506 × 5
  doc_id           user          datetime             size comment
  <chr>            <chr>         <dttm>              <dbl> <chr>
1 Geoffrey Chaucer 86.31.15.58   2021-01-18 09:42:33 68291 /* Origin */
2 Geoffrey Chaucer 86.31.15.58   2021-01-18 09:43:16 68288 /* Taco Bell */
3 Geoffrey Chaucer 88.108.207.22 2021-01-18 11:15:33 68284 /* Career */
4 Geoffrey Chaucer Pahunkat      2021-01-18 11:16:09 68288 Rollback edit(s) by …
5 Geoffrey Chaucer 88.108.207.22 2021-01-18 11:16:40 68293 /* Origin */
# i 501 more rows
```

- `make_datetime(2021)`은 2021년 1월 1일 0시 0분 0초다. 그 뒤의 시각이 모두 남으므로 `>`로 걸러도 2021년의 편집이 남는다.
- 둘째 행의 편집 요약이 `/* Taco Bell */`이다. 14세기 시인의 페이지에 패스트푸드 체인이? 5절에서 다시 본다.

시 단위로 내리기: floor_date()

```
page_revisions |>
  mutate(hour = floor_date(datetime, unit = "hour")) |>
  select(datetime, hour)
```

```
# A tibble: 35,470 × 2
  datetime            hour
  <dttm>              <dttm>
1 2006-07-06 23:24:54 2006-07-06 23:00:00
2 2006-07-24 18:39:08 2006-07-24 18:00:00
3 2006-09-10 13:36:44 2006-09-10 13:00:00
4 2006-09-18 04:59:53 2006-09-18 04:00:00
# i 35,466 more rows
```

내림(floor)

분과 초를 버리고 그 시의 0분 0초로 내린다. 같은 시간대의 편집이 같은 값이 되어 group_by()로 묶을 수 있다.

단위

"minute", "hour", "day", "week", "month", "year". 반올림은 round_date().

하루 단위로 세기: as_date()

```
page_revisions |>
  mutate(date =
    as_date(datetime)) |>
  group_by(date) |>
  summarize(count = n()) |>
  arrange(desc(date))
```

```
# A tibble: 4,831 × 2
  date       count
  <date>     <int>
1 2023-09-30    10
2 2023-09-29    11
3 2023-09-28     9
4 2023-09-27    14
5 2023-09-26    12
# i 4,826 more rows
```

as_date()

<dttm>에서 시각을 떼어 <date>로 만든다. 날짜 단위로 내리는 셈이다.

➔ 4,831일에 걸쳐 편집이 있었다. 하루에 몇 번 고쳐졌는지가 그대로 시계열 하나가 된다.

⚠ 날짜는 **UTC 기준**으로 잘린다. 서울 기준의 날짜가 필요하면 시간대를 먼저 바꾼다(4절).

날짜를 버리고 시각만: as_hms()

```
1  library(hms)
2
3  page_revisions |>
4    mutate(hour = floor_date(
5      datetime, "hour")) |>
6    mutate(hour_hms =
7      as_hms(hour)) |>
8    group_by(hour_hms) |>
9    summarize(count = n()) |>
10   arrange(hour_hms)
```

```
# A tibble: 24 × 2
  hour_hms count
  <time>   <int>
1 00:00     1316
2 01:00     1229
3 02:00     1149
4 03:00     1010
5 04:00      982
# i 19 more rows
```

➔ <time>은 날짜도 시간대도 없이 **하루 안의 시각**만 담는다. 어느 날이든 0시대의 편집은 한데 모인다.

⚠ 교재 코드에는 as_hms(hour) + 1처럼 + 1이 붙어 있다. 그러면 <time>이 <drtn>(시간 길이)으로 바뀌어 1 secs, 3601 secs로 찍힌다. 슬라이드에서는 뺐다.

1 시간을 데이터로

2 날짜로 묻기

3 날짜와 시각

4 **언어와 시간대**
요일 이름의 언어
시간대

5 앞뒤 행과 겹치는 기간

6 마무리

같은 요일, 네 언어

```
page_revisions |>
  mutate(
    dt = datetime,
    en = wday(dt, label = TRUE, locale = "en_US.UTF-8"),
    fr = wday(dt, label = TRUE, locale = "fr_FR.UTF-8"),
    de = wday(dt, label = TRUE, locale = "de_DE.UTF-8"),
    ko = wday(dt, label = TRUE, locale = "ko_KR.UTF-8")
  ) |>
  select(dt, en, fr, de, ko)
```

```
# A tibble: 35,470 × 5
  dt                  en    fr    de    ko
  <dttm>              <ord> <ord> <ord> <ord>
1 2006-07-06 23:24:54 Thu   Jeu   Do    목
2 2006-07-24 18:39:08 Mon   Lun   Mo    월
3 2006-09-10 13:36:44 Sun   Dim   So    일
# i 35,467 more rows
```

➔ `locale`은 언어와 지역의 설정이다. `abbr = FALSE`를 더하면 Thursday, 목요일처럼 온전한 이름이 나온다.

내 화면에는 왜 “월”이 나오는가

설정을 따르는 이름표

`locale`을 주지 않으면 컴퓨터의 설정을 따른다. 영어로 설정된 컴퓨터(Posit Cloud 등)에서는 Mon, 한국어로 설정된 노트북에서는 “월”이 나온다. %b·%a로 만든 그래프 이름표도 마찬가지다.

한 번에 정하기

R Markdown 맨 앞에 한 줄을 두면 누가 돌려도 같은 이름표가 나온다. 교재도 모든 장의 첫머리에서 이렇게 한다.

```
Sys.setlocale("LC_TIME",
              "en_US.UTF-8")
```

➔ 교재가 영어를 권하는 까닭: 오류 메시지도 언어 설정을 따르는데, 영어 메시지로 검색해야 도움을 찾기 훨씬 쉽다.

⚠ 컴퓨터에 그 언어가 설치되어 있지 않으면 `locale`을 준 줄에서 오류가 난다 (OS reports `request to set locale ... cannot be honored`).

💡 이 슬라이드의 출력과 그림은 모두 영어 설정에서 만들었다.

R은 UTC로 적는다

UTC(협정 세계시)

세계 시각의 기준. 런던의 시각과 거의 같지만 서머타임이 없다. `read_csv()`는 시간대 표시가 없는 시각을 UTC로 읽는다.

시간대 이름은 대륙/도시

`Asia/Seoul`, `America/New_York`, `Europe/London`. `OlsonNames()`로 598개를 모두 볼 수 있다.

- ➔ 나라 대신 도시를 쓰는 까닭: 국경과 나라 이름은 바뀌어도 도시 이름은 잘 바뀌지 않는다(교재).
- ➔ 서울은 UTC보다 9시간 빠르다. 뉴욕은 겨울에 5시간, 여름에 4시간 늦다(서머타임). R은 이런 규칙을 날짜마다 알아서 적용한다.
- 💡 한국도 1987–88년 여름에는 서머타임을 썼다. R의 시간대 자료는 이것까지 안다(KDT).

with_tz(): 같은 순간, 다른 시계

```
page_revisions |>
  select(datetime) |>
  mutate(dt_seoul = with_tz(datetime, tzone = "Asia/Seoul")) |>
  mutate(hour_utc = hour(datetime), hour_seoul = hour(dt_seoul),
         dt_diff = datetime - dt_seoul)
```

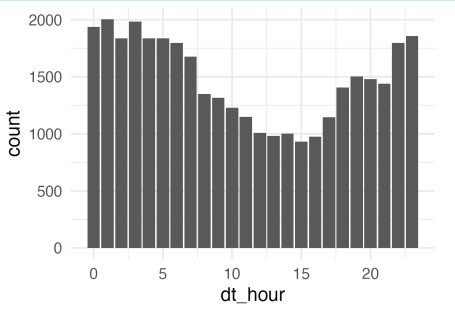
```
# A tibble: 35,470 × 5
  datetime            dt_seoul            hour_utc hour_seoul dt_diff
  <dttm>              <dttm>                 <int>      <int> <drtn>
1 2006-07-06 23:24:54 2006-07-07 08:24:54       23          8 0 secs
2 2006-07-24 18:39:08 2006-07-25 03:39:08       18          3 0 secs
3 2006-09-10 13:36:44 2006-09-10 22:36:44       13         22 0 secs
4 2006-09-18 04:59:53 2006-09-18 13:59:53        4         13 0 secs
# i 35,466 more rows
```

➔ 두 열의 차이(dt_diff)는 0 secs다. R에게는 **같은 순간**이고, 보여 주는 시계만 다르다.

⚠ 첫 행은 UTC로 7월 6일 밤 11시, 서울로는 **7월 7일** 아침 8시다. 날짜와 요일도 바뀌므로 as_date()나 wday()보다 시간대를 먼저 정한다.

몇 시에 고치는가

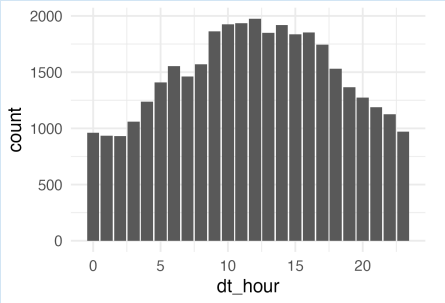
```
page_revisions |>
  mutate(time_local =
    with_tz(datetime,
            "Asia/Seoul")) |>
  mutate(dt_hour =
    hour(time_local)) |>
  group_by(dt_hour) |>
  summarize(count = n()) |>
  ggplot() +
    geom_col(aes(dt_hour,
                 count))
```



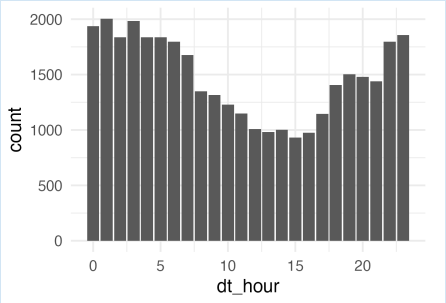
➔ 서울 시각으로 보면 **자정에서 새벽 5시 사이**가 가장 붐비고, 오후 3시 무렵이 가장 한가하다.

같은 자료, 두 시계

뉴욕 시각 (교재)



서울 시각



➔ 뉴욕 시각으로는 **한낮**이 가장 붐빈다. 같은 막대들이 13–14시간 밀린 모양이다. 교재는 저자들이 있는 미국 동부의 시각으로 그렸다.

❗ 서울의 새벽이 뉴욕의 낮이다. 편집자는 주로 아메리카와 유럽에 있다고 **짐작**할 수 있지만, 편집자의 위치는 자료에 없다.

force_tz(): 시계를 잘못 읽었을 때

```
x <- ymd_hms("2026-10-26 10:30:00")    # 서울에서 적은 오전 10시 30분
x
force_tz(x, "Asia/Seoul")
with_tz(x, "Asia/Seoul")
```

```
[1] "2026-10-26 10:30:00 UTC"
[1] "2026-10-26 10:30:00 KST"
[1] "2026-10-26 19:30:00 KST"
```

함수	순간	보이는 시각	쓰는 때
with_tz()	그대로	바뀐다	바르게 적힌 시각을 다른 시계로 볼 때
force_tz()	바뀐다	그대로	시간대 이름표가 잘못 붙었을 때

⚠ 한국에서 기록한 시각도 R로 읽으면 UTC가 붙는다. 서울 시각이라면 force_tz()로 이름표를 고친다. with_tz()를 쓰면 **9시간이 어긋난다**.

1 시간을 데이터로

2 날짜로 묻기

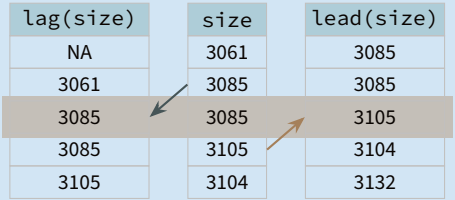
3 날짜와 시각

4 언어와 시간대

5 **앞뒤 행과 겹치는 기간**
윈도우 함수
범위 조인

6 마무리

윈도우 함수: 앞뒤 행을 본다



lag(size)	size	lead(size)
NA	3061	3085
3061	3085	3085
3085	3085	3105
3085	3105	3104
3105	3104	3132

윈도우 함수(window function)

열 하나를 받아 **길이가 같은** 열을 돌려주되, 값을 정할 때 **행의 순서**와 이웃한 행을 본다. `lag()`는 한 행 앞, `lead()`는 한 행 뒤의 값을 가져온다.

➔ `mutate()`의 계산은 한 행 안에서 끝났고, `summarize()`는 여러 행을 한 행으로 줄였다. 윈도우 함수는 행 수를 그대로 두고 **옆 행**을 들여다본다.

⚠ 행의 순서가 곧 시간의 순서여야 한다. 아니라면 `arrange(datetime)`을 먼저 한다.

lag()와 lead()

```
page_revisions |>
  mutate(size_last = lag(size), size_next = lead(size)) |>
  select(doc_id, datetime, size, size_last, size_next)
```

```
# A tibble: 35,470 × 5
  doc_id          datetime             size size_last size_next
  <chr>           <dttm>              <dbl>     <dbl>     <dbl>
1 Marie de France 2006-07-06 23:24:54  3061        NA      3085
2 Marie de France 2006-07-24 18:39:08  3085      3061      3085
3 Marie de France 2006-09-10 13:36:44  3085      3085      3105
4 Marie de France 2006-09-18 04:59:53  3105      3085      3104
5 Marie de France 2006-09-23 22:27:20  3104      3105      3132
# i 35,465 more rows
```

- ➔ 첫 행에는 앞 행이 없으므로 `size_last`가 NA다. 마지막 행의 `size_next`도 그렇다.
- ➔ `lag(size, 2)`처럼 둘째 인자를 주면 두 행 앞을 본다.
- 💡 `tidyverse`를 부를 때 뜨는 `dplyr::lag() masks stats::lag()`는 이제부터 `lag()`가 dplyr의 것이라는 뜻이다.

페이지 경계를 넘는다

```
page_revisions |>
  group_by(doc_id) |>                # 왼쪽 출력은 이 줄과
  mutate(size_last = lag(size)) |>
  ungroup() |>                       # 이 줄을 뺀 결과다
  slice(499:502) |>                  # 첫 페이지의 끝과 둘째 페이지의 처음
  select(doc_id, size, size_last)
```

	doc_id	size	size_last
	<chr>	<dbl>	<dbl>
1	Marie de France	30154	30080
2	Marie de France	30080	30154
3	Geoffrey Chaucer	62881	30080
4	Geoffrey Chaucer	62920	62881

	doc_id	size	size_last
	<chr>	<dbl>	<dbl>
1	Marie de France	30154	30080
2	Marie de France	30080	30154
3	Geoffrey Chaucer	62881	NA
4	Geoffrey Chaucer	62920	62881

⚠ 묶지 않으면(왼쪽) Chaucer 첫 편집의 “바로 앞”이 Marie de France의 마지막 편집(30080)이 된다.

➔ `group_by(doc_id)` 뒤에 쓰면(오른쪽) **묶음 안에서만** 앞뒤를 본다. 경계에는 NA가 생긴다.

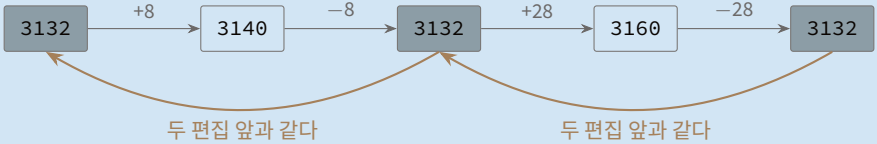
편집마다 늘어난 크기

```
page_revisions |>
  group_by(doc_id) |>
  mutate(size_diff = size - lag(size)) |>
  ungroup() |>
  select(doc_id, datetime, size, size_diff)
```

```
# A tibble: 35,470 × 4
  doc_id          datetime             size size_diff
  <chr>           <dttm>              <dbl>     <dbl>
1 Marie de France 2006-07-06 23:24:54  3061        NA
2 Marie de France 2006-07-24 18:39:08  3085        24
3 Marie de France 2006-09-10 13:36:44  3085         0
4 Marie de France 2006-09-18 04:59:53  3105        20
5 Marie de France 2006-09-23 22:27:20  3104        -1
# i 35,465 more rows
```

- 지금 크기에서 바로 앞 크기를 뺀다. 양수는 덧붙인 바이트, 음수는 지운 바이트다.
- 셋째 편집은 0이다. 크기는 그대로인데 무언가를 고쳤다(글자 하나 바꾸기 등).

되돌림의 모양



되돌림(revert)

누군가의 편집을 취소하고 앞의 판으로 돌려놓는 일. 장난 편집(vandalism)을 지우는 가장 흔한 방법이다.

교재의 규칙

크기가 **두 편집 앞**과 똑같으면 되돌림으로 본다. `size == lag(size, 2)`.

➔ Marie de France 페이지의 2006년 가을 편집 다섯 번이다. 8바이트를 더했다 지우고, 28바이트를 더했다 지운다.

규칙으로 찾기

```
page_revisions |>
  group_by(doc_id) |>
  mutate(size_diff = size - lag(size),
         reversion = (size == lag(size, 2))) |>
  ungroup() |>
  select(doc_id, datetime, size_diff, reversion) |>
  filter(reversion)
```

```
# A tibble: 4,927 × 4
  doc_id          datetime            size_diff reversion
  <chr>           <dttm>                  <dbl> <lgl>
1 Marie de France 2006-10-03 18:19:32        -8 TRUE
2 Marie de France 2006-10-11 05:29:32       -28 TRUE
3 Marie de France 2006-11-16 18:14:53      -118 TRUE
4 Marie de France 2007-05-22 01:56:59         0 TRUE
# i 4,923 more rows
```

➔ 35,470번 가운데 4,927번(약 14%)이 규칙에 걸린다. `reversion`은 논리값(`<lgl>`) 열이라 `filter()`에 그대로 넣는다.

31초 만의 되돌림

```
page_revisions |>
  filter(doc_id == "Geoffrey Chaucer") |>
  mutate(size_diff = size - lag(size)) |>
  filter(as_date(datetime) == ymd("2022-09-29")) |>
  select(datetime, size, size_diff, comment)
```

```
# A tibble: 2 × 4
  datetime             size size_diff comment
  <dttm>              <dbl>     <dbl> <chr>
1 2022-09-29 15:14:19 62957    -11333 /* Life */
2 2022-09-29 15:14:50 74290     11333 Reverted edits by [[Special:Contributions…
```

- ➔ 15시 14분 19초에 익명 사용자가 Chaucer의 생애(Life) 절에서 11,333바이트를 지웠다. **31초 뒤** 다른 사용자가 되돌렸다.
- ➔ 3절의 그림에서 Chaucer의 선이 뚝 떨어졌다 돌아온 곳이 이것이다.
- 💡 걸러 낸 행 수가 2라는 것도 확인했다. 그날 Chaucer 페이지의 편집은 이 둘뿐이다.

규칙이 놓치는 것

```
# A tibble: 7 × 4
  user           time     size_diff reversion
  <chr>          <time>       <dbl> <lgl>
1 86.31.15.58    09:42:33         3 FALSE
2 86.31.15.58    09:43:16        -3 TRUE
3 88.108.207.22  11:15:33        -4 FALSE
4 Pahunkat       11:16:09         4 TRUE
5 88.108.207.22  11:16:40         5 FALSE
6 88.108.207.22  11:17:41        18 FALSE
7 ClueBot NG     11:17:44       -23 FALSE
```

두 번을 한 번에

ClueBot NG(장난 편집을 지우는 봇)가 +5, +18 두 편집을 **한꺼번에** 되돌렸다(-23). 규칙이 놓친다.

- ➔ 2021년 1월 18일의 Chaucer 페이지다. 1–2행은 절 제목 Origin을 Taco Bell로 바꿨다가(+3바이트) 43초 뒤 되돌린 것으로 보인다.
- ⚠ 거꾸로 크기 변화가 0인 편집이 이어져도 걸린다. 4,927건 가운데 578건이 그렇다.
- ❗ 규칙은 **어림**이다. 편집 요약(comment)처럼 다른 열과 맞추어 보며 의심한다.

얼마나 빨리 되돌리나

```
page_revisions |>
  group_by(doc_id) |>
  mutate(size_diff = size - lag(size),
         reversion = (size == lag(size, 2)),
         gap = difftime(datetime, lag(datetime), units = "mins")) |>
  ungroup() |>
  filter(reversion, size_diff != 0) |>
  summarize(n = n(), gap_median = median(gap),
            within_5 = mean(gap <= 5))
```

```
# A tibble: 1 × 3
      n gap_median    within_5
  <int> <drtn>           <dbl>
1  4349 3.766667 mins    0.522
```

- `difftime()`은 두 시각의 간격을 원하는 단위(`"secs"`, `"mins"`, `"hours"`, `"days"`)로 잰다. 되돌린 편집과 바로 앞 편집 사이의 간격이다.
- 크기 변화가 0인 578건을 빼고 4,349건을 보면 중앙값은 **3.8분**, 절반 남짓(52%)이 5분 안에 되돌려졌다.

같음이 아니라 범위로

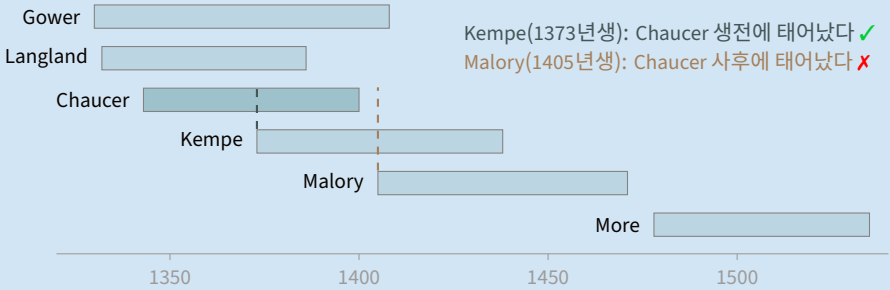
```
periods <- tribble(
  ~period, ~start, ~end,
  "동진",  317,    420,
  "송",    420,    479)
records <- tibble(year =
  c(372, 420, 472))
records |>
  inner_join(periods, join_by(
    between(year, start, end)))
```

```
# A tibble: 4 × 4
   year period start   end
  <dbl> <chr>  <dbl> <dbl>
1   372 동진     317   420
2   420 동진     317   420
3   420 송       420   479
4   472 송       420   479
```

➔ 5강의 조인은 키가 **같은** 행끼리 이었다. 범위 조인은 부등식이나 `between()`으로 **범위에 드는** 행을 잇는다. 10강의 시기 구분이다.

⚠ 420년은 동진의 끝이자 송의 시작이라 **두 행**에 붙었다. 한쪽에만 넣으려면 `join_by(year >= start, year < end)`.

생애가 겹치는 작가



두 사람이 함께 살았다

x가 태어난 뒤, x가 죽기 전에 y가 태어났다: $\text{born}_x \leq \text{born}_y < \text{died}_x$

➔ 부등식이 **둘**이다. Thomas More는 위의 누구와도 겹치지 않는다(Malory가 죽고 7년 뒤에 태어났다).

join_by()에 부등식 둘

```
meta <- read_csv(file.path("hdr", "data", "wiki_uk_meta.csv.gz"))
overlap_pairs <- meta |>
  inner_join(meta,
             join_by(x$born <= y$born, x$died > y$born),
             suffix = c("", "2")) |>    # 겹치는 열 이름의 꼬리
  select(doc_id, doc_id2, born, died, born2, died2)
overlap_pairs
```

```
# A tibble: 767 × 6
  doc_id           doc_id2            born  died born2 died2
  <chr>            <chr>             <dbl> <dbl> <dbl> <dbl>
1 Marie de France  Marie de France    1160  1215  1160  1215
2 Geoffrey Chaucer Geoffrey Chaucer  1343  1400  1343  1400
3 Geoffrey Chaucer Margery Kempe     1343  1400  1373  1438
# i 764 more rows
```

➔ 같은 표를 자기 자신과 잇는다. x$는 왼쪽, y$는 오른쪽 표의 열이다.

⚠ 자기 자신과의 쌍이 74개뿐이다. Seamus Heaney의 몰년이 출생년과 같은 1939년으로 **잘못** 적혀 있다(실제 2013년).

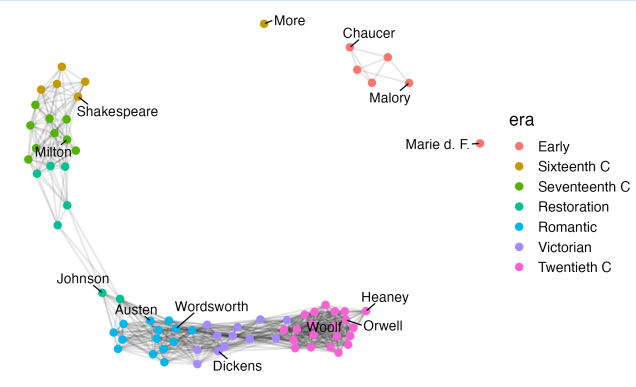
네트워크로 그리기

```
library(hdir)
library(ggrepel)

set.seed(1)
graph <- hdir_network_metrics(select(overlap_pairs, 1:2))
graph$node |>
  inner_join(meta, by = c("id" = "doc_id")) |>
  mutate(era = fct_inorder(era)) |>
  ggplot(aes(x, y)) +
    geom_segment(aes(xend = xend, yend = yend),
                 data = graph$edge, alpha = 0.1) +
    geom_point(aes(color = era)) +
    geom_text_repel(aes(label = id), size = 3, max.overlaps = 10) +
    theme_void()
```

➔ 9강의 코드 그대로다. 함께 살았던 쌍을 에지로 삼고, 점의 색은 meta의 시대 구분(era)이다.

함께 살았던 작가 네트워크



따로 떨어진 셋

Marie de France와 More는 혼자다. Chaucer, Malory 등 초기 작가 다섯은 따로 한 덩어리다.

나머지는 한 띠

16세기에서 20세기까지 **시간 순서대로** 이어진다.

➔ 교재가 정한 시대 구분(색)이 생몰년만으로 만든 네트워크와 잘 맞는다.

❗ 시간 자료로 네트워크를 만들었다. 9강의 방법과 오늘의 방법을 **겹쳐** 쓴 것이다.

1 시간을 데이터로

2 날짜로 묻기

3 날짜와 시각

4 언어와 시간대

5 앞뒤 행과 겹치는 기간

6 **마무리**

시간 자료 앞에서 물을 것

오늘 만난 함정

1. 빠진 날은 없는가? (Charlotte Smith의 9일)
2. 걸렀더니 몇 행이 남았는가? (아무것도 걸러지지 않은 2020년)
3. 평균을 끌어올린 하루는 없는가? (9월 5일의 화요일)
4. 어느 달력, 어느 시간대로 적혔는가? (음력·연호, UTC와 서울)
5. 기록이 커버하는 기간은 같은가? (500번의 편집)
6. 규칙은 무엇을 놓치는가? 경계는 어느 쪽에 넣는가? (ClueBot, 420년)

→ 교재의 결론: 시간은 텍스트·네트워크 같은 다른 방법과 겹쳐 쓸 때 해석을 두텁게 한다. 그래도 어떤 계산도 다른 연구와 사료 없이 홀로 서지는 않는다.

오늘의 함수

함수	하는 일	패키지
`make_date()`·`make_datetime()`	수 열에서 날짜·시각 만들기	lubridate
`ymd()`·`mdy()`·`dmy()`·`ymd_hms()`	글자에서 날짜·시각 읽기	lubridate
`wday()`·`hour()`	요일·시 꺼내기	lubridate
`scale_x_date()`·`scale_x_datetime()`	날짜 축의 눈금과 이름표	ggplot2
`floor_date()`·`as_date()`	단위 내리기 / 날짜만 남기기	lubridate
`as_hms()`	하루 안의 시각만 남기기	hms
`with_tz()`·`force_tz()`	시계 바꾸기 / 시간대 이름표 고치기	lubridate
`difftime()`	두 시각의 간격	base R
`lag()`·`lead()`	앞·뒤 행의 값	dplyr
`join_by()`	부등식 `between()`으로 잇기	dplyr

날짜 함수는 대부분 `lubridate`에 있다. 치트시트 — `https://rstudio.github.io/cheatsheets/lubridate.pdf`

오늘 배운 것

날짜 만들기

- 행 번호·소수 연도의 한계
- `make_date()`, `ymd()`로 <date>
- 달력에 없는 날, 읽지 못한 날은 NA

시간대와 언어

- UTC로 적고 `with_tz()`로 본다
- 틀린 이름표는 `force_tz()`
- 요일 이름은 `locale`을 따른다

묶고 거르기

- 요일·시·하루 단위로 묶기
- 행 수와 평균 뒤의 하루를 확인

앞뒤 행과 범위

- `lag()`·`lead()`는 묶음 안에서
- 되돌림 규칙은 어림이다
- 범위 조인은 경계에 주의한다

확인 질문

스스로 답해 보기

1. 행 번호를 가로축으로 써도 되는 조건 두 가지는 무엇인가?
2. `make_date(2023, 2, 30)`은 무엇을 돌려주는가? 왜 위험한가?
3. “화요일에 조회 수가 가장 많다”는 결론을 의심할 근거를 두 가지 들어 보라.
4. `with_tz()`와 `force_tz()`는 각각 무엇을 바꾸고 무엇을 그대로 두는가?
5. `group_by()` 없이 `lag()`를 쓰면 어디에서 틀린 값이 나오는가?
6. `between()`으로 시기를 붙였더니 행이 늘었다. 왜인가?

💡 답이 막히는 항목이 있으면 eTL Q&A 게시판에 질문을 올린다 (참여 점수 1점).

다음 시간

- 12강 (2026-10-28 수) — 시계열 데이터 사례 연습
  - 온라인 '성지' 게시물에 달린 소원 댓글이 언제 시작해 얼마나 퍼지고 언제 사그라들었는지 따라가 본다.
  - 심형준 (2023), 「성지 밈(聖地 meme) 소원 댓글의 유행과 온라인 종교문화상의 함의」, 『종교연구』 83(2).
  - 오늘의 `floor_date()`로 댓글을 월·주 단위로 묶고, 8강의 텍스트 분석과 겹쳐 쓴다.

더 보고 싶다면

- `lubridate` 문서 — `https://lubridate.tidyverse.org/`
- Wickham, Çetinkaya-Rundel & Grolemund, *R for Data Science* (2판, 2023), 17장 Dates and times — `https://r4ds.hadley.nz/datetimes.html`
- Shumway & Stoffer, *Time Series Analysis and Its Applications: With R Examples* (2017)